

# Fork e branch

---

De uma linha isolada de trabalho a uma proposta revisável.

Repositório, ponteiro,  
sincronização e *pull request*



## ***Fork*, clone e *branch* criam objetos diferentes**

---

### **FORK**

#### **Outro repositório**

Nasce no GitHub  
e pertence à conta  
de Ana.

### **CLONE**

#### **Repositório local**

Traz histórico e  
arquivos para o  
computador.

### **BRANCH**

#### **Ponteiro móvel**

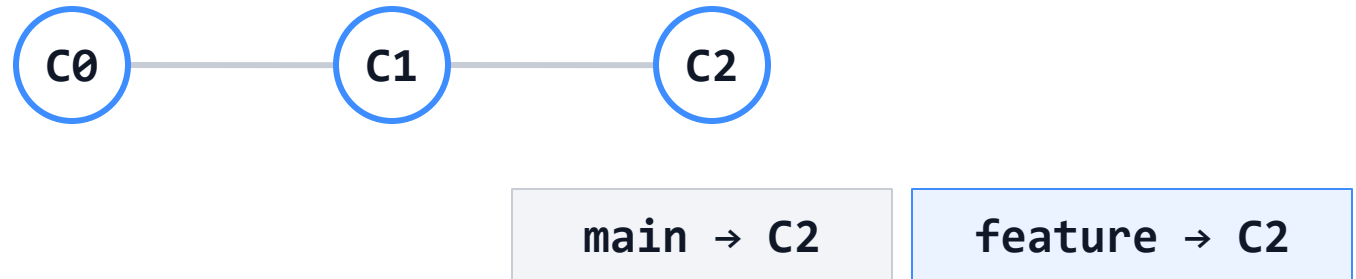
Aponta para um  
*commit* dentro do  
repositório.

**A operação correta depende do objeto que precisamos criar.**

# HEAD decide qual branch avança

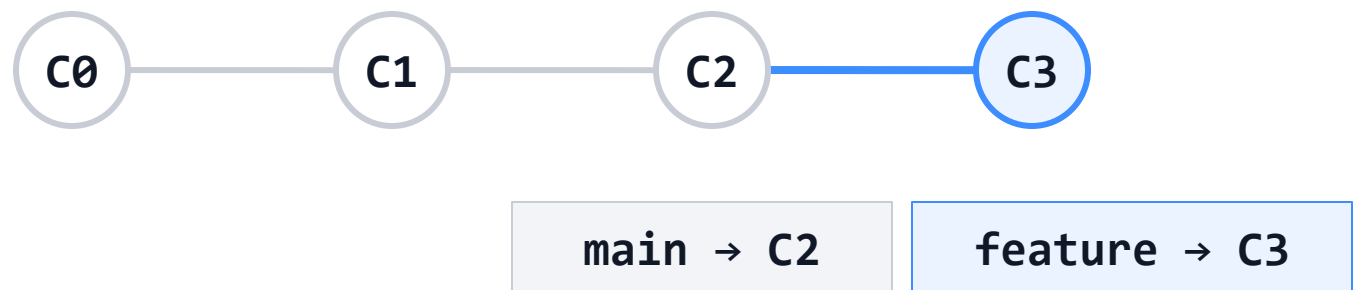
## 1. Ao criar a branch

```
git switch -c feature/ana
```

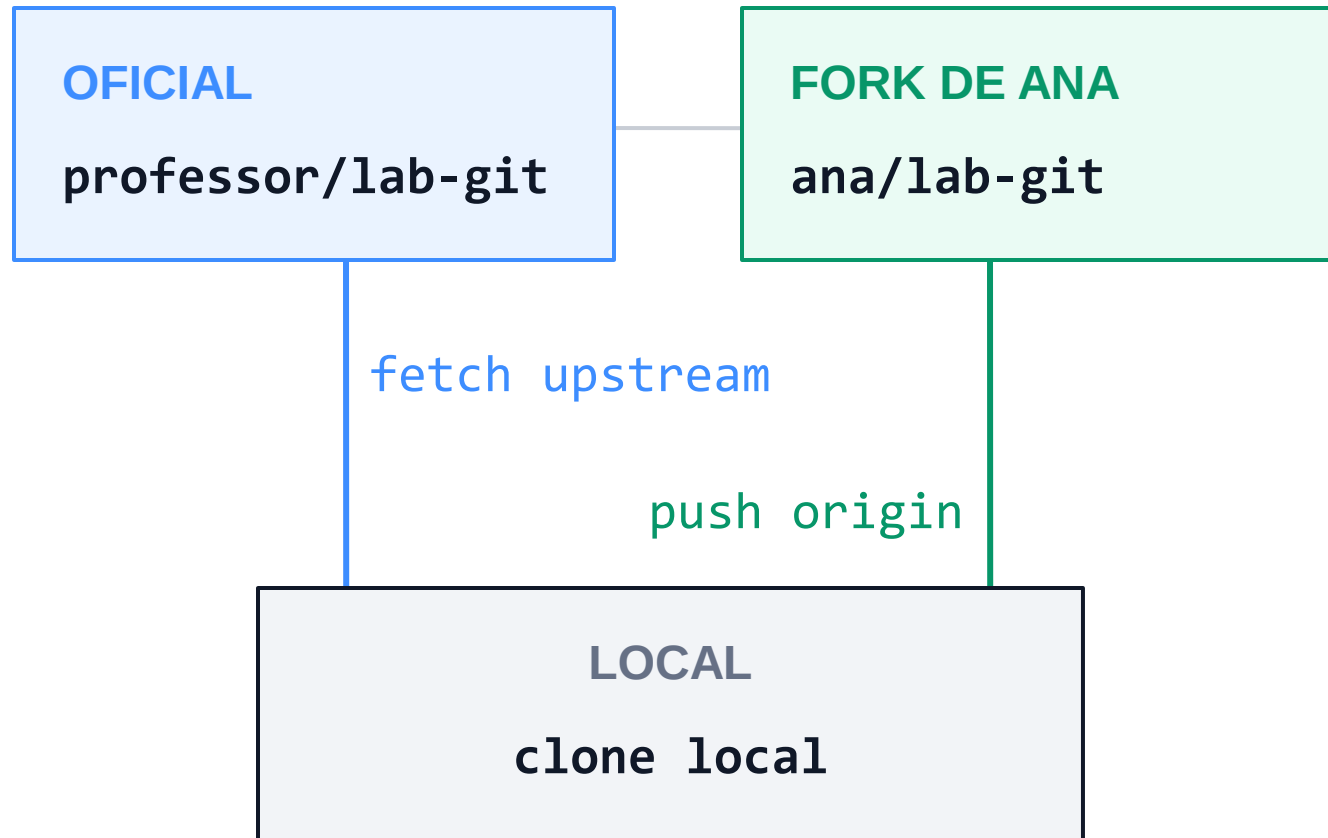


## 2. Depois do commit

```
git commit -m Adiciona-Ana
```



# Origin e upstream ligam três repositórios



## Quatro referências chamadas main

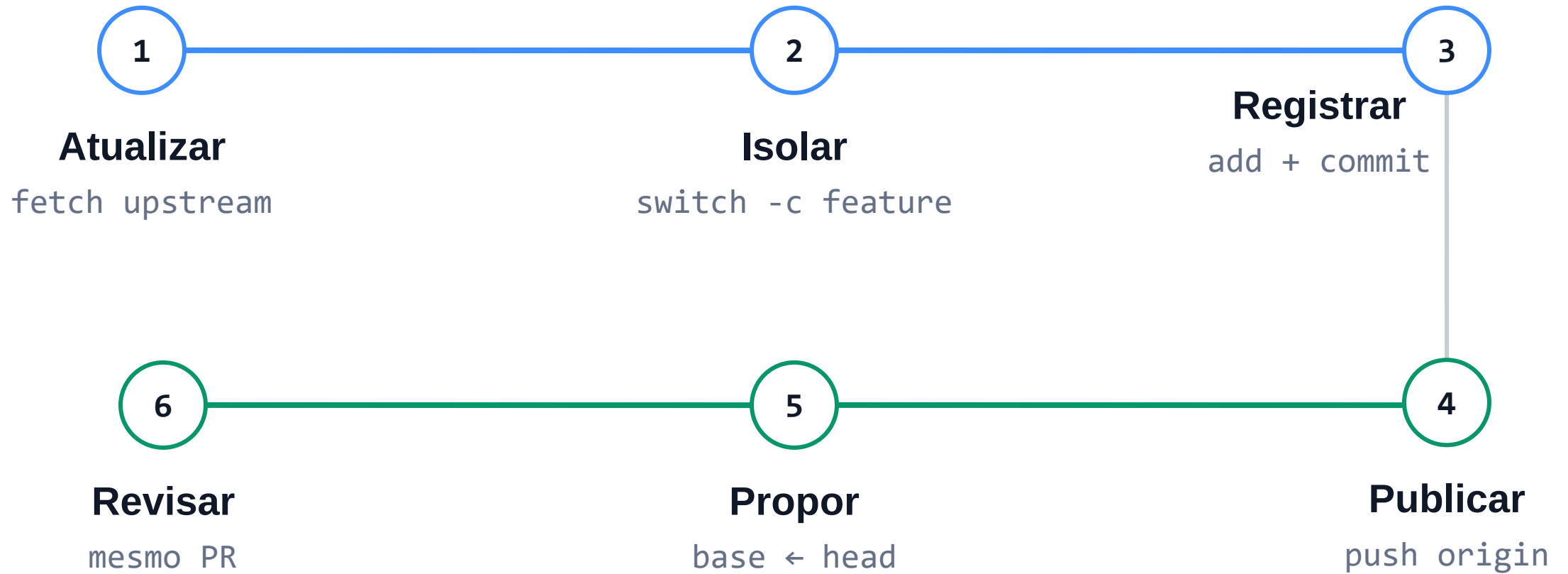
**main oficial**  
versão aceita

**main do fork**  
base de Ana

**main local**  
branch de base

**upstream/main**  
último fetch

# Uma contribuição percorre seis decisões



**Publicar uma branch não integra a main oficial.**

# O pull request liga head, revisão e base



# Quando a *main* avança, sincronize duas vezes

---

## 1. Atualizar a main pessoal

A base local e a main do fork voltam a espelhar o oficial.

```
git switch main  
git fetch upstream  
git merge --ff-only  
upstream/main  
git push origin main
```

## 2. Atualizar a branch aberta

A tarefa recebe a base recente antes do teste e da revisão.

```
git fetch upstream  
git switch feature/ana  
git merge upstream/main  
git push
```

**Conflito pede uma decisão humana; não indica corrupção.**

# Dois fluxos nativos de colaboração no Git

---

## MESMO OBJETIVO, DOIS TRANSPORTES

O mantenedor revisa uma contribuição sem conceder escrita direta na branch principal.

A diferença é o que atravessa o e-mail.

### 1 `git request-pull`

**Viaja:** o pedido

**Código:** repositório acessível

**Final:** `fetch + merge`

### 2 patches por e-mail

**Viaja:** o próprio código

**Artefato:** arquivos `.patch`

**Final:** `git am`



# No request-pull, o código fica no repositório

```
git request-pull <branch-base> <url-do-repositório>
```

## 1 PUBLICAR

### Branch acessível

Os commits permanecem no servidor do colaborador.

## 2 PEDIR

### Mensagem pronta

Resumo, commits e URL são enviados por e-mail.

## 3 INTEGRAR

### fetch • revisar • merge

O mantenedor busca o código, analisa e decide localmente.

**O pedido viaja por e-mail;  
o código permanece no repositório do colaborador.**

# Com patches, o próprio código viaja por e-mail

---

## **git format-patch**

Cria um arquivo de texto por commit. Preserva autoria, data, mensagem e diferenças do código.

## **git send-email**

Envia os patches à lista. A revisão acontece em uma thread, com comentários no texto.

## **git am**

Lê a mensagem aprovada e cria o commit, mantendo os dados do autor original.

1 GERAR

2 DISCUTIR

3 APLICAR

Plataformas web automatizam o transporte, a revisão visual e a integração no servidor.

# Três perguntas evitam os erros mais comuns

---

Antes de qualquer comando, localize o trabalho:

**01**

**Em qual repositório  
estou?**

oficial, *fork* ou clone local

**02**

**Qual branch estou  
alterando?**

*main* ou *branch* da tarefa

**03**

**Para onde a  
mudança vai?**

*origin*, *upstream* ou base do  
PR

